

Îndrumar de laborator

Programare Web (specializarea Tehnologia Informației) Dezvoltarea Aplicațiilor Web (specializarea Informatică, IFR)

Template de proiect: PHP Symfony (backend) + React (frontend) + PostgreSQL, în Docker

Repository: <https://github.com/cezarpapara/daw-template-ifr2-2026-2027>

Despre acest îndrumar

Îndrumarul descrie ce vom construi la laborator, ce programe trebuie instalate, cum se pregătește proiectul o singură dată, cum se lucrează la fiecare laborator și cum este organizat proiectul pe care îl primiți. Este scris pentru studenți fără experiență anterioară: fiecare comandă este însoțită de locul în care se rulează, de motivul pentru care se rulează și de ce ar trebui să se întâmple după ea.

Cum se citește. Capitolele 1–2 explică scopul și tehnologiile. Capitolele 3–5 se parcurg o singură dată, în ordine (instalare, terminal, configurare). Capitolul 6 descrie modul de lucru de la fiecare laborator. Capitolele 7–9 sunt consultate atunci când aveți nevoie de ele. Anexele conțin rezolvări de erori și referințe.

Cuprins

- 1. Obiectivul disciplinei
 - 1.1 Ce vom construi
 - 1.2 Cum se lucrează în industrie: noțiunile de bază
 - 1.3 Tehnologiile folosite și de ce
- 2. Template-ul de proiect
- 3. Conturi și programe de instalat
 - 3.1 Contul de GitHub
 - 3.2 Docker Desktop
 - 3.3 Git și Git Bash
 - 3.4 Editorul de cod
 - 3.5 DBeaver
 - 3.6 Postman
 - 3.7 Verificarea instalărilor
- 4. Terminalul
- 5. Configurarea inițială (o singură dată)
 - 5.1 Configurarea Git
 - 5.2 Clonarea proiectului
 - 5.3 Deschiderea proiectului în editor
 - 5.4 Branch-ul personal
 - 5.5 Prima pornire a backend-ului
 - 5.6 Prima pornire a frontend-ului
- 6. Metodologia de lucru
 - 6.1 La începutul și la sfârșitul fiecărui laborator

- 6.2 Scrierea codului
- 6.3 Salvarea lucrului: commit și push
- 6.4 Preluarea actualizărilor de pe main
- 6.5 Conflicte
- 7. Lucrul cu baza de date
 - 7.1 Unde sunt datele
 - 7.2 Conectarea din DBeaver
 - 7.3 Adăugarea unei tabele noi
- 8. Lucrul cu API-ul
 - 8.1 Ce este un API
 - 8.2 Cererile disponibile în template
 - 8.3 Testarea din Swagger
 - 8.4 Testarea din Postman
- 9. Structura proiectului, fișier cu fișier
 - 9.1 Imaginea de ansamblu
 - 9.2 Fișierele din rădăcină
 - 9.3 Backend-ul
 - 9.4 Frontend-ul
 - 9.5 Drumul unei cereri prin proiect
- Anexa A. Docker pe Windows: Hyper-V sau WSL 2
- Anexa B. Probleme frecvente
- Anexa C. Un proiect nou, de la zero
- Anexa D. Tutoriale și documentație

1. Obiectivul disciplinei

1.1 Ce vom construi

Scopul laboratorului este să învățați cum se construiește o aplicație web **așa cum se face astăzi în firmele de software**: nu un singur fișier care face totul, ci un sistem format din mai multe părți, fiecare cu rolul ei, care comunică între ele prin reguli clare.

Veți lucra pe un proiect concret: un **magazin online simplu**, cu categorii și produse. Proiectul este format din două aplicații separate:

- **backend-ul**: partea care rulează pe server, păstrează datele și le oferă la cerere;
- **frontend-ul**: partea pe care o vede utilizatorul în browser.

La final veți ști să porniți un astfel de proiect, să adăugați tabele în baza de date, să scrieți adrese noi în API, să afișați date în pagini React și să lucrați cu Git în echipă. Proiectul opțional pentru nota de laborator este o aplicație la alegere, construită cu aceleași tehnologii.

1.2 Cum se lucrează în industrie: noțiunile de bază

Mai jos sunt noțiunile cu care veți lucra, explicate pe scurt. Le veți reîntâlni în tot îndrumarul.

Backend. Programul care rulează pe server, „în spate”. Primește cereri (de exemplu „dă-mi lista de produse”), le prelucerează, citește sau scrie în baza de date și trimite înapoi un răspuns. Utilizatorul nu îl vede niciodată direct. În proiectul nostru, backend-ul este scris în **PHP**, cu

framework-ul **Symfony**.

Frontend. Aplicația care rulează în browserul utilizatorului: paginile, butoanele, formularele. Frontend-ul nu păstrează date; le cere de la backend și le afișează. Al nostru este scris în **JavaScript**, cu biblioteca **React**.

API. „Limba” în care vorbesc frontend-ul și backend-ul. Un API este o listă de adrese web (de exemplu `/api/products`) la care se pot trimite cereri. Fiecare adresă răspunde cu date în format **JSON**, un text structurat, ușor de citit atât de programe, cât și de oameni. Separarea backend-frontend prin API permite ca aceleași date să fie folosite și de un site, și de o aplicație de telefon.

Baza de date. Locul în care backend-ul păstrează datele, în **tabele** (ca niște foi de calcul: coloane cu tipuri fixe, rânduri cu înregistrări). Folosim **PostgreSQL**, una dintre cele mai răspândite baze de date.

Framework. Un set de componente gata scrise, peste care construieți aplicația, ca să nu scrieți de la zero lucruri pe care le are orice aplicație (rute, conexiunea la baza de date, configurare). Symfony este framework-ul pentru backend; React este biblioteca pentru frontend.

Docker. Un program care pornește aplicațiile în **containere**: mici „cutii” izolate, fiecare cu propriul program instalat în interior (una cu PHP, una cu baza de date etc.). Avantajul: proiectul merge identic pe orice calculator, fără să instalați PHP, Node.js sau PostgreSQL pe laptopul vostru.

Git și GitHub. **Git** este programul care ține evidența modificărilor din cod: fiecare salvare (numită *commit*) rămâne în istoric și se poate reveni la ea. **GitHub** este site-ul unde se păstrează proiectul, ca să poată lucra mai mulți oameni pe el. În firme, nimeni nu trimite cod pe e-mail: totul trece prin Git.

Branch. O „ramură” de lucru în Git: o copie a codului în care lucrați fără să afectați copia principală. Fiecare student va avea branch-ul lui.

1.3 Tehnologiile folosite și de ce

Partea	Tehnologie	De ce
Backend	PHP 8.5 + Symfony 8.1	PHP este cel mai folosit limbaj pentru web; Symfony este unul dintre cele două framework-uri PHP standard în industrie.
Baza de date	PostgreSQL 18	Bază de date relațională gratuită, robustă, folosită pe scară largă.
Lucrul cu baza de date	Doctrine ORM	Permite lucrul cu tabelele prin clase PHP, fără a scrie SQL de mână pentru operațiile obișnuite.
Documentația API	Swagger (NelmioApiDocBundle)	Generează automat o pagină cu toate adresele API, din care se pot și testa.
Server web	Nginx	Primește cererile din browser și le trimite către PHP.
Frontend	React 19 + Vite	React este cea mai folosită bibliotecă pentru interfețe web; Vite pornește rapid aplicația în modul de dezvoltare.
Rulare	Docker + Docker Compose	Pornește toate componentele, configurate identic, cu o singură comandă.

Partea	Tehnologie	De ce
Versionare	Git + GitHub	Standardul în industrie pentru păstrarea și partajarea codului.

2. Template-ul de proiect

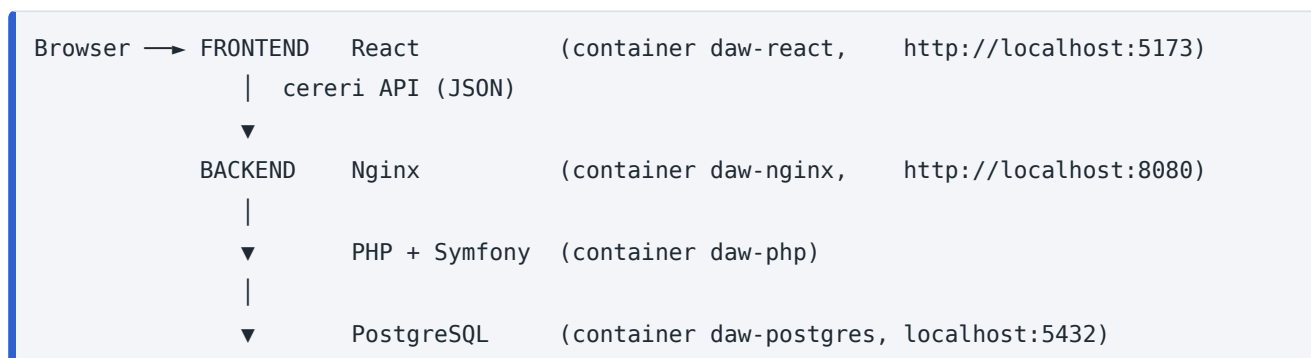
Pentru a nu pierde timp cu configurări, primiți un **template**: un proiect deja configurat și funcțional, de la care porniți. Se găsește la adresa

<https://github.com/cezarpapara/daw-template-ifr2-2026-2027>

Template-ul conține:

- backend-ul Symfony, cu două tabele (**category** și **product**), câteva date de exemplu și un API pentru listare, adăugare și ștergere;
- pagina de documentație a API-ului (**Swagger**), din care puteți testa fiecare cerere;
- frontend-ul React, cu patru pagini: **Acasă**, **Categorii**, **Produse** și **Detalii produs**;
- configurarea Docker pentru toate componentele;
- o colecție de cereri de exemplu pentru Postman;
- acest îndrumar ([Ghid-DAW.pdf](#)).

Componentele și adresele la care le găsiți după pornire:



Fiecare `daw-...` este un container Docker. Pornirea lor este descrisă în [capitolul 5](#).

3. Conturi și programe de instalat

Instalați programele în ordinea de mai jos. Fiecare subcapitol spune ce face programul, la ce îl folosim, de unde se descarcă și unde găsiți un tutorial.

3.1 Contul de GitHub

Ce este. GitHub este site-ul pe care se păstrează codul proiectelor și prin care colaborează programatorii. Template-ul este publicat acolo, iar codul vostru va fi urcat tot acolo.

Ce aveți de făcut. Creați un cont la <https://github.com/signup>.

- Alegeți un **username** care conține numele vostru, de forma `prenume-nume` (de exemplu `ion-popescu`); GitHub nu acceptă punct în username.
- În **Settings** → **Public profile** → **Name** treceți numele complet.
- Folosiți o adresă de e-mail pe care o citiți: pe ea veți primi invitația de acces la proiect.

Documentație: <https://docs.github.com/en/get-started>

3.2 Docker Desktop

Ce este. Programul care creează și pornește containerele. În proiect avem patru containere (server web, PHP, baza de date, React); Docker Desktop le pornește pe toate, dintr-un fișier de configurare, cu o singură comandă.

La ce îl folosim. La fiecare laborator, pentru a porni și a opri aplicația. Trebuie să fie **pornit** ori de câte ori lucrați la proiect.

Descărcare: <https://www.docker.com/products/docker-desktop/>

- **Windows:** înainte de instalare, sistemul trebuie pregătit (virtualizare, Hyper-V sau WSL 2). Urmăriți pașii din [Anexa A](#), apoi reveniți aici.
- **macOS:** descărcați varianta pentru procesorul vostru (Apple Silicon sau Intel; o vedeți în meniul → About This Mac) și instalați normal.

După instalare, porniți Docker Desktop și așteptați să afișeze „**Engine running**” în colțul din stânga jos.

Tutoriale: <https://www.youtube.com/watch?v=pTFZFxd4hOI> (introducere, 1 oră) · <https://www.youtube.com/watch?v=3c-iBn73dDE> (curs complet) **Documentație:** <https://docs.docker.com/get-started/>

3.3 Git și Git Bash

Ce este. Git ține evidența modificărilor din cod și le trimite pe GitHub. Pe Windows, instalarea lui aduce și **Git Bash**: un terminal (fereastră de comenzi) în care funcționează toate comenzile din acest îndrumar.

La ce îl folosim. La descărcarea proiectului, la salvarea lucrului (commit), la urcarea lui pe GitHub (push) și la preluarea actualizărilor.

Descărcare: <https://git-scm.com/downloads>. La instalare lăsați opțiunile implicite.

Tutoriale: <https://www.youtube.com/watch?v=8JJ101D3knE> (1 oră) · în română: <https://www.youtube.com/watch?v=iVpuVkVeBcw> **Documentație:** <https://git-scm.com/docs>

3.4 Editorul de cod

Ce este. Programul în care scrieți codul. Colorează sintaxa, semnalează greșelile și are un terminal încorporat, pe care îl vom folosi.

Alegeți una dintre variante:

- **Visual Studio Code** (gratuit): <https://code.visualstudio.com/download>. La prima deschidere a proiectului, VS Code propune câteva extensii utile (Docker, PHP Intelephense, DotENV); apăsați **Install**.
- **PhpStorm** (pentru backend) și **WebStorm** (pentru frontend), de la JetBrains: produse plătite, dar **gratuite pentru studenți**. Licența se activează la <https://www.jetbrains.com/academy/student-pack/>.

Documentație: <https://code.visualstudio.com/docs> · <https://www.jetbrains.com/help/phpstorm/>

3.5 DBeaver

Ce este. Un program cu interfață grafică pentru baze de date: vă arată tabelele, datele din ele și vă lasă să rulați interogări SQL.

La ce îl folosim. Să verificăm ce se întâmplă în baza de date după ce rulăm comenzi sau folosim aplicația (de exemplu, dacă produsul adăugat din React a ajuns în tabelă).

Descărcare: <https://dbeaver.io/download/> (alegeți **DBeaver Community**, varianta pentru sistemul vostru).

Tutorial: <https://www.youtube.com/watch?v=GgODmMBVUD4> (conectare la PostgreSQL și interogări) **Documentație:** <https://dbeaver.com/docs/dbeaver/>

3.6 Postman

Ce este. Un program cu care trimiteți cereri către un API și vedeți răspunsul, fără să scrieți cod. Este folosit zilnic de programatorii de backend pentru a-și testa munca.

La ce îl folosim. Să testăm adresele din API ([capitolul 8](#)). Proiectul conține o colecție de cereri gata făcute, pe care o importați în Postman.

Descărcare: <https://www.postman.com/downloads/> (este nevoie de un cont gratuit).

Tutorial: <https://www.youtube.com/watch?v=VywxIQ2ZXw4> **Documentație:** <https://learning.postman.com/docs/>

3.7 Verificarea instalărilor

Deschideți un terminal (pe Windows **Git Bash**, din meniul Start; pe macOS **Terminal**) și rulați, pe rând:

```
docker --version
docker compose version
git --version
```

Fiecare comandă trebuie să afișeze o versiune. Dacă apare `command not found`, programul respectiv nu este instalat corect (sau, pentru Docker, nu este pornit).

4. Terminalul

Majoritatea problemelor de la început nu vin din cod, ci din rularea unei comenzi **în alt loc** decât trebuie. Capitolul acesta explică de ce și cum evitați situația.

Ce este terminalul. O fereastră în care scrieți comenzi text, iar calculatorul le execută. Pe Windows folosim **Git Bash**; pe macOS, aplicația **Terminal**. Editorul de cod are și el un terminal încorporat ([capitolul 5.3](#)); pe acela îl vom folosi cel mai des.

Terminalul se află mereu într-un folder. Imaginați-vă terminalul ca pe o persoană care stă într-o cameră. Când îi spuneți „deschide cutia”, ea caută cutia în camera în care se află. La fel, o comandă caută fișierele în folderul în care „stă” terminalul. Dacă terminalul este în alt folder, comanda nu găsește fișierele și afișează o eroare.

Cum vedeți în ce folder sunteți. În Git Bash, deasupra locului în care scrieți apare un rând de forma:

```
ion@LAPTOP MINGW64 /c/proiecte/daw-template-ifr2-2026-2027 (popescu.ion)
$
```

- `/c/proiecte/daw-template-ifr2-2026-2027` este folderul curent (adică `C:\proiecte\daw-template-ifr2-2026-2027`);
- `(popescu.ion)` este branch-ul Git pe care lucrați ([capitolul 5.4](#)); apare doar în folderele cu Git;
- după `$` scrieți comanda. Când copiați comenzi din îndrumar, **nu copiați și semnul `$`**.

Comenzile de orientare:

```
pwd          # afiseaza folderul in care se afla terminalul
ls           # afiseaza fisierele si folderele din folderul curent
cd nume      # intra in folderul "nume"
cd ..        # iese un nivel mai sus
```

Textul de după # este un comentariu explicativ; nu se scrie.

Regula proiectului. Proiectul este configurat astfel încât **toate comenzile din acest îndrumar să se ruleze din folderul principal al proiectului**, `daw-template-ifr2-2026-2027`. Îl recunoașteți simplu: dacă scrieți `ls`, vedeți `README.md`, `Ghid-DAW.pdf`, `backend`, `frontend` și `postman`. Există două excepții, marcate explicit: configurarea Git (5.1), care se face din orice folder, și clonarea (5.2), care se face din folderul `proiecte`.

De ce funcționează totul din folderul principal:

- comenzile **git** lucrează pe folderul curent și pe tot ce este sub el; din folderul principal cuprind întregul proiect;
- comenzile **docker compose** din îndrumar indică explicit fișierul de configurare (`-f backend/compose.yaml`), iar această cale este scrisă pornind din folderul principal;
- comenzile **PHP** (de exemplu `composer install`) nu pot rula direct pe laptop, pentru că PHP nu este instalat acolo; le trimitem în containerul de PHP cu prefixul `docker exec -it daw-php`, tot din folderul principal.

În îndrumar, înaintea fiecărui grup de comenzi, găsiți:

- **Unde:** terminalul și folderul din care se rulează;
- **De ce:** ce rezolvă comanda;
- **Rezultat:** ce ar trebui să vedeți după rulare.

5. Configurarea inițială (o singură dată)

Pașii din acest capitol se fac **o singură dată**, la început (sau din nou doar dacă instalați proiectul pe alt calculator). Parcurgeți-i în ordine.

5.1 Configurarea Git

De ce. Fiecare commit are trecut autorul (nume și e-mail). Dacă Git nu știe cine sunteți, refuză primul commit cu mesajul `Please tell me who you are`.

Unde. Git Bash (pe macOS, Terminal), din **orice folder**: setarea este valabilă pentru tot laptopul.

Înlocuiți numele și e-mailul cu ale voastre; e-mailul trebuie să fie **cel al contului de GitHub**:

```
git config --global user.name "Ion Popescu"
git config --global user.email "ion.popescu@exemplu.com"
git config --global pull.rebase false
```

A treia comandă stabilește modul în care Git combină modificările la `git pull`, ca să nu întrebe de fiecare dată.

Rezultat. `git config --global user.name` afișează numele vostru; `git config --global user.email` afișează e-mailul.

5.2 Clonarea proiectului

De ce. „A clona” înseamnă a descărca proiectul de pe GitHub pe calculator, împreună cu tot istoricul lui Git.

Unde. Git Bash / Terminal, din folderul în care vreți să stea proiectul. Recomandăm un folder simplu, `proiecte`: **nu** pe Desktop, **nu** într-un folder sincronizat cu OneDrive sau iCloud și fără spații sau diacritice în nume, pentru că Docker poate avea probleme cu astfel de căi.

Mai întâi creați folderul și intrați în el.

Pe Windows:

```
mkdir -p /c/proiecte
cd /c/proiecte
```

Pe macOS:

```
mkdir -p ~/proiecte
cd ~/proiecte
```

(`mkdir -p` creează folderul; dacă există deja, nu face nimic. `cd` mută terminalul în el.)

Terminalul se află acum în `proiecte`. Clonați proiectul și intrați în folderul lui:

```
git clone https://github.com/cezarpapara/daw-template-ifr2-2026-2027.git
cd daw-template-ifr2-2026-2027
ls
```

Rezultat. `ls` afișează `README.md Ghid-DAW.pdf backend frontend postman`. Vă aflați în folderul principal al proiectului. Clonarea nu se repetă; dacă rulați din nou `git clone` în același loc, primiți eroarea `destination path already exists`.

5.3 Deschiderea proiectului în editor

De ce. De acum înainte lucrați din editor. Dacă deschideți folderul potrivit, terminalul încorporat al editorului pornește direct în folderul principal al proiectului.

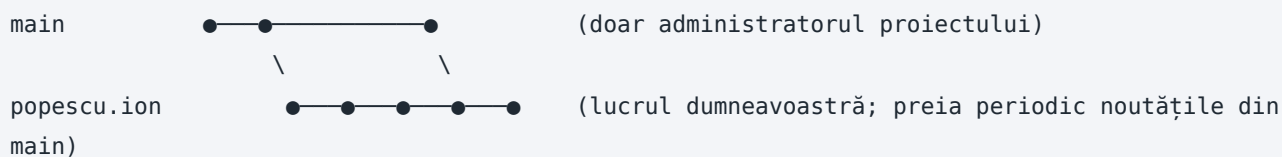
1. În editor: **File → Open Folder** (VS Code) sau **File → Open** (PhpStorm, WebStorm). Selectați **exact** folderul `daw-template-ifr2-2026-2027`: nu `proiecte`, nu `backend`, nu `frontend`.
2. Deschideți terminalul încorporat: în VS Code, **Terminal → New Terminal**; pe Windows, din săgeata de lângă **+** alegeți **Git Bash**. În PhpStorm / WebStorm, apăsați tab-ul **Terminal** din partea de jos.
3. Scrieți `pwd`.

Rezultat. Calea afișată se termină cu `daw-template-ifr2-2026-2027`. Toate comenzile care urmează se rulează din acest terminal.

5.4 Branch-ul personal

De ce. Proiectul de pe GitHub este comun pentru toți studenții. Dacă toată lumea ar modifica aceeași copie, modificările s-ar suprascrie una pe alta. De aceea:

- branch-ul **main** este copia principală; pe el lucrează **doar administratorul proiectului** (cadru didactic);
- fiecare student lucrează pe **branch-ul lui**, numit `nume.prenume`; ce faceți acolo nu afectează pe nimeni.



Unde. Terminalul din editor, în folderul principal.

Înlocuiți `popescu.ion` cu numele vostru: litere mici, fără diacritice, cu punct între nume și prenume.

```
git switch -c popescu.ion
git push -u origin popescu.ion
```

- prima comandă creează branch-ul (`-c`, de la *create*) și vă mută pe el;
- a doua îl urcă pe GitHub (`origin` este numele sub care Git ține minte adresa proiectului de pe GitHub; `-u` leagă branch-ul local de cel de pe GitHub, astfel încât pe viitor ajunge simplul `git push`). La prima rulare se deschide browserul pentru autentificarea în contul de GitHub.

Rezultat. `git branch --show-current` afișează numele branch-ului vostru, iar pe GitHub, în lista de branch-uri a proiectului, apare `popescu.ion`.

Dacă primiți eroarea 403 (Permission denied), nu aveți încă drept de scriere în proiect.

Trimiteți cadrului didactic username-ul sau e-mailul de GitHub. Veți primi o invitație pe e-mail; acceptați-o, apoi rulați din nou `git push -u origin popescu.ion`.

5.5 Prima pornire a backend-ului

De ce. Pornim containerele backend-ului, instalăm bibliotecile de care are nevoie Symfony și creăm baza de date cu tabelele ei.

Unde. Terminalul din editor, în folderul principal. **Docker Desktop trebuie să fie pornit.**

Rulați comenzile **una câte una**, așteptând să se termine fiecare.

Pasul 1 - pornirea containerelor:

```
docker compose -f backend/compose.yaml up -d --build
```

`-f backend/compose.yaml` indică fișierul de configurare; `up` pornește containerele; `-d` le lasă să ruleze în fundal; `--build` construiește imaginea de PHP (necesar doar prima dată și când se modifică fișierele din `backend/docker`). Prima rulare durează câteva minute, pentru că se descarcă imaginile de bază.

Pasul 2 - instalarea bibliotecilor PHP:

```
docker exec -it daw-php composer install
```

`docker exec -it daw-php` înseamnă „rulează comanda următoare în containerul `daw-php`”.
`composer install` descarcă bibliotecile enumerate în `backend/app/composer.json` în folderul `backend/app/vendor`.

Pasul 3 - crearea bazei de date și a tabelelor:

```
docker exec -it daw-php php bin/console doctrine:database:create --if-not-exists
docker exec -it daw-php php bin/console doctrine:migrations:migrate
```

Prima comandă creează baza de date (dacă nu există deja). A doua rulează **migrările**: fișierele din `backend/app/migrations`, care creează tabelele `category` și `product` și adaugă câteva date de exemplu. La întrebarea afișată, scrieți `yes` și apăsați Enter.

Rezultat. În Docker Desktop, la **Containers**, apare grupul `daw-backend` cu `daw-nginx`, `daw-php` și `daw-postgres`, toate cu starea *Running*. În browser:

- `http://localhost:8080/api/health` afișează `"message": "Salut din Symfony!"` și `"database": "conectata"`;
- `http://localhost:8080/api/doc` afișează documentația API (Swagger).

5.6 Prima pornire a frontend-ului

De ce. Pornim aplicația React, partea vizibilă în browser.

Unde. Același terminal, în folderul principal.

```
docker compose -f frontend/compose.yaml up -d --build
```

Prima pornire durează 1-2 minute, cât se instalează pachetele JavaScript în container.

Rezultat. La <http://localhost:5173> se deschide aplicația. Pe pagina **Acasă** apare o casetă **verde**, care confirmă că frontend-ul comunică cu backend-ul. Pe paginile **Categorii** și **Produse** vedeți datele de exemplu și puteți adăuga sau șterge înregistrări.

Configurarea inițială s-a încheiat. De acum, la fiecare laborator urmați [capitolul 6](#).

6. Metodologia de lucru

6.1 La începutul și la sfârșitul fiecărui laborator

Unde. Toate comenzile: terminalul din editor, în folderul principal.

La început:

1. Porniți Docker Desktop și așteptați „Engine running”.
2. Deschideți proiectul în editor ([capitolul 5.3](#)).

3. Rulați:

```
git switch popescu.ion
docker compose -f backend/compose.yaml up -d
docker compose -f frontend/compose.yaml up -d
```

Prima comandă vă asigură că lucrați pe branch-ul vostru. Următoarele două pornesc containerele; fără `--build`, pornirea durează câteva secunde.

La sfârșit:

```
docker compose -f backend/compose.yaml stop
docker compose -f frontend/compose.yaml stop
```

Containerele se opresc; datele din baza de date și codul rămân.

6.2 Scrierea codului

Ambele aplicații rulează în **modul de dezvoltare**: codul de pe laptop este „văzut” direct în containere, deci modificările se aplică fără nicio repornire.

- **Backend** (fișierele din `backend/app`): salvați fișierul și reîncărcați pagina sau cererea.
- **Frontend** (fișierele din `frontend/app/src`): salvați fișierul; pagina din browser se actualizează singură.

Singurul caz în care este nevoie de `--build` din nou este modificarea fișierelor din `backend/docker` sau `frontend/docker`.

Unde scrieți cod nou: adrese API noi în `backend/app/src/Controller`, tabele noi în `backend/app/src/Entity` ([capitolul 7.3](#)), pagini React noi în `frontend/app/src/pages` (și adăugarea lor în `App.jsx`).

6.3 Salvarea lucrului: commit și push

De ce. Git nu salvează nimic automat. Un **commit** este o „fotografie” a codului la un moment dat, însoțită de un mesaj. **Push** urcă fotografiile pe GitHub, pe branch-ul vostru. Faceți commit ori de câte ori ați terminat un lucru (o pagină, o funcționalitate): este mai ușor de urmărit și de reparat.

Unde. Terminalul din editor, în folderul principal.

```
git status
git add .
git commit -m "Am adaugat pagina de contact"
git push
```

- `git status` afișează branch-ul curent (trebuie `0n branch popescu.ion`) și fișierele modificate;
- `git add .` pregătește pentru commit toate modificările din folderul curent și de sub el (de aceea contează să fiți în folderul principal);
- `git commit -m "..."` creează commit-ul, cu un mesaj scurt care spune **ce** ați făcut;
- `git push` urcă commit-urile pe GitHub.

Rezultat. `git status` afișează `nothing to commit, working tree clean`, iar pe GitHub, pe branch-ul vostru, apare commit-ul nou.

6.4 Preluarea actualizărilor de pe main

De ce. Pe parcursul semestrului, cadrul didactic va adăuga lucruri noi pe `main` și vă va anunța. Ca să le aveți și voi, trebuie să le „trageți” pe branch-ul vostru.

Unde. Terminalul din editor, în folderul principal.

Pasul 1 - salvați ce aveți lucrat, ca modificările voastre să nu se amestece cu cele noi:

```
git add .
git commit -m "Ce am lucrat pana acum"
git push
```

(Dacă nu doriți să păstrați modificările curente, le anulați cu `git reset --hard .`) `git status` trebuie să afișeze `working tree clean`.

Pasul 2 - aduceți noutățile și combinați-le cu branch-ul vostru:

```
git switch main
git pull origin main
git switch popescu.ion
git merge main --no-edit
git push
```

Pe rând: treceți pe `main`; descărcați noutățile de pe GitHub; reveniți pe branch-ul vostru; combinați (*merge*) noutățile cu codul vostru (`--no-edit` acceptă mesajul de commit propus automat); urcați rezultatul.

Pasul 3 - dacă noutățile au adus biblioteci sau tabele noi, rulați și:

```
docker exec -it daw-php composer install
docker exec -it daw-php php bin/console doctrine:migrations:migrate
docker compose -f frontend/compose.yaml restart
```

Dacă la `git merge` apare cuvântul `CONFLICT`, continuați cu [capitolul 6.5](#).

6.5 Conflicte

Ce este un conflict. Git combină automat modificările atunci când ele sunt în locuri diferite. Dacă **aceeași linie** a fost modificată și pe `main`, și pe branch-ul vostru, Git nu poate decide singur ce variantă să păstreze și vă cere să alegeți. Nu se pierde nimic. Dacă doriți să renunțați și să reveniți la starea dinaintea merge-ului: `git merge --abort`.

Cum arată. În fișierul cu probleme apar marcate:

```
<<<<<<< HEAD
    <h1>Magazinul lui Ion</h1>
=====
    <h1>Aplicatie DAW - versiunea 2</h1>
>>>>>>> main
```

Între `<<<<<<< HEAD` și `=====` este varianta voastră; între `=====` și `>>>>>>> main` este varianta de pe `main`.

Cum se rezolvă.

1. Deschideți fișierul în editor. Deasupra marcajelor, editorul afișează butoanele **Accept Current Change** (varianta voastră), **Accept Incoming Change** (varianta de pe `main`) și **Accept Both Changes** (amândouă). Alegeți una sau scrieți manual varianta corectă.
2. Verificați că nu au rămas linii cu `<<<<<<`, `=====`, `>>>>>>`. Salvați.
3. În terminal, în folderul principal:

```
git add .
git commit --no-edit
git push
```

Alte situații:

Mesajul	Explicație și rezolvare
Your local changes ... would be overwritten	Aveți modificări nesalvate. Faceți commit (6.4, pasul 1) și repetați comanda.
conflict în <code>package-lock.json</code> sau <code>composer.lock</code>	Fișierele acestea se generează automat; nu le editați. Păstrați varianta de pe <code>main</code> : <code>git checkout --theirs calea/fisierului</code> , apoi <code>git add .</code> și <code>git commit --no-edit</code> .
ați lucrat din greșală pe <code>main</code> , fără commit	<code>git switch popescu.ion</code> : modificările vin cu voi pe branch-ul vostru, unde faceți commit.
s-a deschis în terminal un editor necunoscut (Vim)	Tastați <code>:wq</code> și apăsați Enter.

7. Lucrul cu baza de date

7.1 Unde sunt datele

Containerul `daw-postgres` scrie datele în folderul `backend/docker/postgres/data` din proiect. Ele rămân acolo și după oprirea containerelor. Folderul este trecut în `.gitignore`, deci **nu** se urcă pe GitHub: fiecare student are propria bază de date.

Pentru a reface baza de date de la zero (toate datele se pierd), din folderul principal:

```
docker compose -f backend/compose.yaml down
rm -rf backend/docker/postgres/data
docker compose -f backend/compose.yaml up -d
docker exec -it daw-php php bin/console doctrine:migrations:migrate
```

7.2 Conectarea din DBeaver

De ce. Ca să vedeți tabelele și datele direct, nu doar prin aplicație.

Backend-ul trebuie să fie pornit (capitolul 6.1).

1. Deschideți DBeaver. În meniu: **Database → New Database Connection**.
2. Alegeți **PostgreSQL** și apăsați **Next**.
3. Completați:

Câmp	Valoare
Host	<code>localhost</code>

Câmp	Valoare
Port	5432
Database	daw
Username	daw
Password	daw

4. Apăsați **Test Connection**. La prima utilizare, DBeaver cere să descarce driverul PostgreSQL: acceptați. Trebuie să apară **Connected**.

5. Apăsați **Finish**.

Rezultat. În panoul din stânga, la **daw** → **Schemas** → **public** → **Tables**, apar tabelele `category`, `product` și `doctrine_migration_versions` (în ultima, Doctrine ține evidența migrărilor rulate). Dublu-click pe o tabelă, apoi tab-ul **Data**, afișează înregistrările.

Pentru o interogare SQL: click dreapta pe conexiune → **SQL Editor** → **Open SQL script**, scrieți de exemplu `SELECT * FROM product;` și apăsați **Ctrl + Enter**.

7.3 Adăugarea unei tabele noi

Noțiuni. În Symfony nu se creează tabelele direct în SQL, ci prin **entități**: clase PHP care descriu o tabelă (numele ei, coloanele și tipurile lor). Din entități, Doctrine generează **migrări**: fișiere cu comenzile SQL care aduc baza de date la forma descrisă. Avantajul: structura bazei de date este în cod, versionată cu Git, iar colegii o pot reproduce rulând migrările.

Unde. Terminalul din editor, în folderul principal.

Pasul 1 - creați entitatea:

```
docker exec -it daw-php php bin/console make:entity
```

Comanda pune întrebări: numele entității (de exemplu `Review`), apoi, pentru fiecare coloană, numele, tipul (`string`, `integer`, `text`, `decimal`, `datetime` etc.) și dacă poate fi goală. Apăsați Enter la un nume gol ca să terminați. Rezultatul este fișierul `backend/app/src/Entity/Review.php` (și `backend/app/src/Repository/ReviewRepository.php`).

Pasul 2 - generați migrarea:

```
docker exec -it daw-php php bin/console doctrine:migrations:diff
```

Doctrine compară entitățile cu baza de date și scrie diferențele, ca SQL, într-un fișier nou din `backend/app/migrations`. Deschideți-l și citiți-l: veți vedea comanda `CREATE TABLE`.

Pasul 3 - aplicați migrarea:

```
docker exec -it daw-php php bin/console doctrine:migrations:migrate
```

Rezultat. În DBeaver, după **Refresh** (F5), apare tabela nouă. Faceți commit la fișierele create (entitatea, repository-ul și migrarea), ca să ajungă pe GitHub.

Alte comenzi utile: `doctrine:migrations:status` (ce migrări au fost rulate), `debug:router` (toate adresele API ale aplicației); ambele cu prefixul `docker exec -it daw-php php bin/console`.

8. Lucrul cu API-ul

8.1 Ce este un API

Backend-ul expune un **API REST**: un set de adrese (numite *endpoint-uri*) la care se trimit cereri HTTP. O cerere are:

- o **metodă**, care spune ce vrem: `GET` (citește), `POST` (creează), `PUT / PATCH` (modifică), `DELETE` (șterge);
- o **adresă**, de exemplu `http://localhost:8080/api/products`;
- opțional, un **corp** (*body*) cu date în format JSON, de exemplu `{"name": "Mouse", "price": "59.90", "categoryId": 1}`.

Răspunsul are un **cod de stare** (200 OK, 201 creat, 204 șters, 400 date greșite, 404 negăsit) și, de obicei, un corp JSON cu datele cerute.

8.2 Cererile disponibile în template

Toate adresele încep cu `http://localhost:8080`.

Metodă	Adresă	Ce face	Corp (JSON)
GET	<code>/api/health</code>	verifică backend-ul și baza de date	-
GET	<code>/api/categories</code>	lista categoriilor	-
POST	<code>/api/categories</code>	adaugă o categorie	<code>{"name": "Jucarii"}</code>
GET	<code>/api/products</code>	lista produselor	-
GET	<code>/api/products/{id}</code>	un produs, după id	-
POST	<code>/api/products</code>	adaugă un produs	<code>{"name": "Mouse", "description": "optic", "price": "59.90", "categoryId": 1}</code>
DELETE	<code>/api/products/{id}</code>	șterge un produs	-

8.3 Testarea din Swagger

Swagger este o pagină generată automat din codul backend-ului, care listează toate endpoint-urile și permite testarea lor din browser. Nu necesită instalare.

- Deschideți <http://localhost:8080/api/doc>.
- Apăsați pe un endpoint, de exemplu **GET /api/products**, apoi pe **Try it out**.
- Apăsați **Execute**. Mai jos apar codul de răspuns și corpul JSON.
- Pentru **POST /api/categories**: după **Try it out**, modificați textul JSON din căsuța *Request body* (de exemplu `{"name": "Jucarii"}`) și apăsați **Execute**. Răspunsul 201 confirmă crearea. Verificați în DBeaver sau la **GET /api/categories**.

8.4 Testarea din Postman

Importarea colecției din proiect.

- Deschideți Postman. Apăsați **Import** (stânga sus).
- Alegeți fișierul `postman/DAW-API.postman_collection.json` din folderul proiectului (sau trageți-l în fereastră).

3. În panoul din stânga apare colecția **DAW API**, cu cererile grupate pe *Categorii* și *Produse*.
4. Deschideți **Lista produselor** și apăsați **Send**. Răspunsul JSON apare jos, cu codul **200 OK**.

Crearea unei cereri manual (POST).

1. Apăsați **+** pentru o cerere nouă.
2. Alegeți metoda **POST** din lista derulantă și scrieți adresa `http://localhost:8080/api/products`.
3. Apăsați tab-ul **Body**, alegeți **raw**, iar din lista din dreapta **JSON**.
4. Scrieți corpul:

```
{
  "name": "Tastatura mecanica",
  "description": "Switch-uri rosii",
  "price": "349.00",
  "categoryId": 1
}
```

5. Apăsați **Send**. Răspunsul **201 Created** conține produsul creat, cu **id**-ul lui.

Ștergerea (DELETE). Cerere nouă, metoda **DELETE**, adresa

`http://localhost:8080/api/products/5` (înlocuiți **5** cu **id**-ul produsului). Răspunsul **204 No Content** confirmă ștergerea; un **id** inexistent întoarce **404**.

9. Structura proiectului, fișier cu fișier

9.1 Imaginea de ansamblu

daw-template-ifr2-2026-2027/	folderul principal (de aici se rulează comenzile)
├─ README.md	descriere scurtă, trimite la acest îndrumar
├─ Ghid-DAW.pdf	acest îndrumar
├─ .gitignore, .gitattributes	reguli Git pentru tot proiectul
├─ .vscode/extensions.json	extensiile recomandate pentru VS Code
├─ postman/	colecția de cereri pentru Postman
├─ backend/	APLICATIA 1: API-ul Symfony
│ └─ compose.yaml	containerele backend-ului
│ └─ docker/	configurarea containerelor (nginx, php, postgres)
│ └─ app/	codul Symfony
└─ frontend/	APLICATIA 2: aplicația React
└─ compose.yaml	containerul frontend-ului
└─ docker/	configurarea containerului (react)
└─ app/	codul React

Cele două aplicații sunt **independente**: fiecare are propriul `compose.yaml`, propriul folder `docker` cu configurarea containerelor și propriul folder `app` cu cod. Singura legătură dintre ele este API-ul.

9.2 Fișierele din rădăcină

- **README.md** – pagina afișată de GitHub când deschideți proiectul; conține o descriere scurtă și linkul către acest îndrumar.

- **Ghid-DAW.pdf** – acest îndrumar.
- **.gitignore** – lista fișierelor pe care Git le ignoră (nu le urcă pe GitHub): fișiere de sistem, setări ale editorului. Fiecare aplicație mai are câte un **.gitignore** propriu.
- **.gitattributes** – îi spune lui Git să salveze fișierele text cu terminații de linie Linux, ca să funcționeze corect în containere indiferent de sistemul de operare.
- **.vscode/extensions.json** – lista extensiilor pe care VS Code le propune la deschiderea proiectului.
- **postman/DAW-API.postman_collection.json** – cererile de exemplu pentru Postman ([capitolul 8.4](#)).

9.3 Backend-ul

9.3.1 backend/compose.yaml

Fișierul citit de Docker Compose. Descrie cele trei containere ale backend-ului și legăturile dintre ele:

- **daw-nginx** – serverul web Nginx. Ascultă pe portul **8080** al laptopului și trimite cererile de tip PHP către **daw-php**. Vede folderul **backend/app** (doar în citire) ca să poată servi fișierele din **public**.
- **daw-php** – containerul cu PHP, construit din **docker/php/Dockerfile**. Folderul **backend/app** de pe laptop este „montat” în container la **/var/www/app**: orice modificare din editor se vede imediat în container.
- **daw-postgres** – baza de date PostgreSQL 18, cu utilizatorul, parola și numele bazei **daw**. Portul **5432** este expus pe laptop pentru DBeaver, iar datele sunt salvate în **docker/postgres/data**.

9.3.2 backend/docker/

- **nginx/default.conf** – configurarea Nginx: rădăcina site-ului este **app/public**; orice adresă care nu corespunde unui fișier existent este trimisă la **index.php** (punctul de intrare în Symfony), prin containerul **daw-php**, pe portul 9000.
- **php/Dockerfile** – „rețeta” imaginii de PHP: pornește de la imaginea oficială **php:8.5-fpm**, instalează extensiile necesare (**pdo_pgsql** pentru PostgreSQL, **intl**, **zip**), copiază Composer și setările din **php.ini**.
- **php/php.ini** – setări PHP pentru dezvoltare: fusul orar, limitele de memorie și de încărcare, afișarea erorilor, recitirea imediată a fișierelor modificate.
- **postgres/** – conține **README.md** (explicații) și, după prima pornire, folderul **data/** cu baza de date (ignorat de Git).

9.3.3 backend/app/ : aplicația Symfony

Folderul cu codul propriu-zis. Structura este cea standard a oricărui proiect Symfony, deci ce învățați aici se aplică la orice proiect Symfony.

Fișiere de configurare și dependențe

- **.env** – variabilele de mediu ale aplicației: modul de rulare (**APP_ENV=dev**), conexiunea la baza de date (**DATABASE_URL**, care indică utilizatorul **daw**, parola **daw**, containerul **daw-postgres** și baza **daw**) și adresele din care frontend-ul are voie să apeleze API-ul (**CORS_ALLOW_ORIGIN**). Fișierul se urcă pe GitHub, deci nu conține secrete reale. Pentru setări personale se creează **.env.local** (ignorat de Git), ale cărui valori le înlocuiesc pe cele din **.env**.

- **composer.json** – lista bibliotecilor PHP de care depinde aplicația (Symfony, Doctrine, Nelmio etc.) și versiunile acceptate. **composer.lock** fixează versiunile exacte instalate, ca toți colegii să aibă aceleași. **symfony.lock** ține evidența „rețetelor” de configurare aplicate de Symfony Flex la instalarea pachetelor.
- **.editorconfig** – reguli de formatare (indentare, terminații de linie) pe care editorul le aplică automat.
- **.gitignore** – ignoră `vendor/`, `var/` și fișierele `.env.local`.

Punctele de intrare

- **public/index.php** – singurul fișier accesibil din browser. Orice cerere HTTP ajunge aici, iar el pornește aplicația Symfony (`Kernel`), care decide ce controller răspunde.
- **bin/console** – punctul de intrare pentru comenzile din terminal (`php bin/console ...`): migrări, generarea entităților, afișarea rutelor.

config/ : configurarea framework-ului

- **bundles.php** – lista componentelor (*bundle-uri*) activate: Symfony, Doctrine, migrările, Nelmio (Swagger și CORS), MakerBundle (generatoare de cod, doar în dezvoltare), Twig.
- **services.yaml** – spune Symfony să înregistreze automat clasele din `src/` ca servicii și să le lege între ele (*autowiring*): de aceea un controller primește, de exemplu, un repository ca parametru, fără cod de legătură.
- **routes.yaml** și **routes/** – de unde se citesc rutele (adresele API): din atributele `#[Route]` scrise pe metodele controller-elor; în `routes/nelmio_api_doc.yaml` sunt definite adresele Swagger (`/api/doc` , `/api/doc.json`).
- **packages/** – câte un fișier de configurare pentru fiecare componentă: `doctrine.yaml` (conexiunea la bază și unde sunt entitățile), `doctrine_migrations.yaml` (unde sunt migrările), `nelmio_cors.yaml` (regulile CORS), `nelmio_api_doc.yaml` (titlul și descrierea documentației API), `framework.yaml`, `routing.yaml`, `cache.yaml`, `twig.yaml`, `property_info.yaml` (setări standard, pe care nu trebuie să le modificați).
- **preload.php**, **reference.php** – fișiere generate de Symfony, pentru performanță și pentru completarea automată din editor; nu se modifică.

src/ : codul aplicației (aici lucrați cel mai mult)

- **Kernel.php** – clasa care pornește aplicația; nu se modifică.
- **Controller/** – clasele care răspund la cereri. Fiecare metodă publică cu atributul `#[Route('/adresa', methods: ['GET'])]` devine un endpoint:
 - `HealthController.php` – / (redirecționează la Swagger) și `/api/health`;
 - `CategoryController.php` – listarea și adăugarea categoriilor;
 - `ProductController.php` – listarea, afișarea, adăugarea și ștergerea produselor; conține și validări simple (nume obligatoriu, preț numeric, categorie existentă).
- **Entity/** – entitățile, adică descrierea tabelelor: `Category.php` (tabela `category`: `id`, `name`) și `Product.php` (tabela `product`: `id`, `name`, `description`, `price`, legătura `category`). Relația este „o categorie are mai multe produse” (*one-to-many*). Fiecare entitate are o metodă `toArray()`, care pregătește datele pentru răspunsul JSON.
- **Repository/** – clasele prin care se citesc datele din tabele: `CategoryRepository.php` și `ProductRepository.php` (acesta conține și un exemplu de interogare proprie, `findAllWithCategory()`, care aduce produsele împreună cu categoria lor dintr-o singură interogare).

Alte foldere

- **migrations/** – migrările bazei de date, în ordinea rulării: `Version20261003120000.php` creează tabelele; `Version20261003120100.php` adaugă datele de exemplu. Fiecare migrare are o metodă `up()` (ce se execută) și `down()` (cum se anulează).
- **templates/** – șabloane HTML Twig; conține doar `base.html.twig`, folosit de pagina Swagger. API-ul nostru nu generează HTML.
- **vendor/** – bibliotecile instalate de Composer (apare după `composer install`; ignorat de Git).
- **var/** – fișiere temporare generate de Symfony: cache și loguri (`var/log/dev.log` este primul loc în care căutați o eroare; ignorat de Git).

9.4 Frontend-ul

9.4.1 frontend/compose.yaml

Describe containerul `daw-react`, construit din `docker/react/Dockerfile`. Expune portul `5173` și montează folderul `frontend/app` în container, cu excepția lui `node_modules`, care rămâne în container (pachetele instalate pe Linux nu sunt compatibile cu Windows sau macOS).

9.4.2 frontend/docker/react/Dockerfile

„Rețeta” containerului: pornește de la imaginea oficială `node:24-alpine`, instalează pachetele din `package.json` și, la fiecare pornire, rulează `npm run dev`, adică serverul de dezvoltare Vite.

9.4.3 frontend/app/ : aplicația React

- **package.json** – lista pachetelor JavaScript folosite (`react`, `react-dom`, `react-router`, `vite` etc.) și comenzile disponibile (`npm run dev`, `npm run build`). **package-lock.json** fixează versiunile exacte, ca `composer.lock` la backend.
- **.env** – conține `VITE_API_URL=http://localhost:8080`, adresa backend-ului. Variabilele care încep cu `VITE_` sunt disponibile în cod prin `import.meta.env`. Pentru setări personale: `.env.local`, ignorat de Git.
- **vite.config.js** – configurarea Vite: portul `5173`, acceptarea conexiunilor din afara containerului și urmărirea modificărilor prin *polling* (necesar pe Windows cu Docker).
- **index.html** – singura pagină HTML. Conține un `<div id="root">` gol, în care React desenează toată aplicația, și încarcă `src/main.jsx`.
- **.oxlintrc.json** – reguli pentru verificarea codului (*linter*).
- **public/favicon.svg** – iconița din tab-ul browserului.
- **node_modules/** – pachetele instalate (în container; ignorat de Git).

src/ : codul React

- **main.jsx** – punctul de pornire: „montează” componenta `App` în `<div id="root">` și activează navigarea între pagini (`BrowserRouter`).
- **App.jsx** – structura comună a tuturor paginilor: meniul de sus și **rutele**, adică legătura dintre adresa din browser și pagina afișată (`/` → `HomePage`, `/categorii` → `CategoriesPage`, `/produse` → `ProductsPage`, `/produse/:id` → `ProductDetailsPage`). O pagină nouă se adaugă aici, cu o linie `<Route>`.
- **api.js** – toate cererile către backend, într-un singur loc: `getProducts()`, `createProduct()`, `deleteProduct()` etc. Fiecare folosește funcția `fetch` a browserului, cu adresa din `.env`. Un endpoint nou în backend înseamnă o funcție nouă aici.
- **App.css** – stilurile vizuale (culori, spațieri, tabele, formulare).

- **pages/** – câte un fișier pentru fiecare pagină:
 - `HomePage.jsx` – apelează `/api/health` și afișează caseta verde sau roșie;
 - `CategoriesPage.jsx` – listează categoriile și are un formular de adăugare;
 - `ProductsPage.jsx` – listează produsele, cu formular de adăugare și buton de ștergere;
 - `ProductDetailsPage.jsx` – afișează un singur produs, al cărui `id` vine din adresă.

Fiecare pagină urmează același tipar: ține datele în **stare** (`useState`), le încarcă de la backend la deschiderea paginii (`useEffect`) și le afișează în HTML (JSX). Studiați `CategoriesPage.jsx`, cel mai simplu exemplu complet, înainte de a scrie o pagină nouă.

9.5 Drumul unei cereri prin proiect

Ce se întâmplă când deschideți pagina **Produse**:

1. Browserul încarcă pagina React de la `daw-react` (<http://localhost:5173/produse>). `App.jsx` vede adresa `/produse` și afișează `ProductsPage`.
2. `ProductsPage` apelează `api.getProducts()`, care trimite `GET` <http://localhost:8080/api/products>.
3. Cererea ajunge la `daw-nginx` (portul 8080). Nginx vede că nu este un fișier static și o trimite la `daw-php`, la `public/index.php`.
4. Symfony caută ruta `/api/products` cu metoda `GET` și o găsește în `ProductController::list()`.
5. Controller-ul cere datele de la `ProductRepository`, care, prin Doctrine, trimite o interogare SQL către `daw-postgres`.
6. Rândurile din tabelă devin obiecte `Product`, apoi, prin `toArray()`, un răspuns JSON cu codul `200`.
7. Răspunsul se întoarce pe același drum. `ProductsPage` îl pune în stare, iar React redesenează tabelul cu produse.

Pentru adăugarea unui produs, drumul este același, cu `POST` și un corp JSON, iar la pasul 5 Doctrine execută un `INSERT`.

Anexa A. Docker pe Windows: Hyper-V sau WSL 2

Pe Windows, Docker rulează într-un mic Linux virtual, care poate fi pornit în două moduri. Alegeți în funcție de ediția de Windows (**Settings** → **System** → **About** → **Edition**):

- **Windows Pro / Education / Enterprise** → **Hyper-V** (recomandat; cel mai stabil în practică);
- **Windows Home** → **WSL 2** (Hyper-V nu este disponibil pe Home).

Pasul comun: virtualizarea. Deschideți Task Manager (Ctrl + Shift + Esc) → **Performance** → **CPU**. Trebuie să apară **Virtualization: Enabled**. Dacă apare **Disabled**: reporniți laptopul, intrați în BIOS/UEFI (de obicei tasta F2, F10, Del sau Esc, apăsată imediat la pornire) și activați **Intel Virtualization Technology (VT-x)** sau, pe procesoare AMD, **SVM Mode**. Salvați și reporniți.

Varianta Hyper-V

1. Apăsați Start și căutați „**Turn Windows features on or off**” („Activare sau dezactivare caracteristici Windows”).
2. Bifați **Hyper-V** și **Containers**, apăsați OK și reporniți.
3. Instalați Docker Desktop. Pe ecranul **Configuration** al instalării, **debifați** „Use WSL 2 instead of Hyper-V (recommended)”.

Varianta WSL 2

1. Click dreapta pe Start → **Terminal (Admin)** și rulați `wsl --install`. Comanda activează WSL și instalează distribuția Ubuntu.
2. Reporniți. Se deschide Ubuntu și cere un nume de utilizator și o parolă (le alegeți voi; la parolă caracterele nu se afișează).
3. Verificați cu `wsl -l -v`: Ubuntu trebuie să apară cu VERSION 2. La nevoie, `wsl --update`.
4. Instalați Docker Desktop. Pe ecranul **Configuration**, **lăsați bifat** „Use WSL 2 instead of Hyper-V (recommended)”.

Documentație: <https://docs.docker.com/desktop/setup/install/windows-install/> · <https://learn.microsoft.com/en-us/windows/wsl/install>

Anexa B. Probleme frecvente

Mesajul sau simptomul	Cauza și rezolvarea
<code>fatal: not a git repository</code>	Terminalul nu este în folderul principal. Verificați cu <code>pwd</code> (capitolul 4).
<code>no configuration file provided sau backend/compose.yaml: no such file</code>	Comanda <code>docker compose</code> nu este rulată din folderul principal.
<code>composer: command not found / php: command not found</code>	Lipsește prefixul <code>docker exec -it daw-php</code> din fața comenzii.
<code>the input device is not a TTY</code>	Apare în Git Bash deschis separat, pe Windows. Rulați comanda din terminalul editorului sau adăugați <code>winpty</code> în față.
<code>Cannot connect to the Docker daemon</code>	Docker Desktop nu este pornit.
<code>No such container: daw-php</code>	Backend-ul nu este pornit (capitolul 6.1).
<code>port is already allocated</code>	Alt program folosește portul 8080, 5173 sau 5432 (de exemplu un PostgreSQL instalat local). Închideți-l.
<code>Dependencies are missing</code>	Nu s-a rulat <code>composer install</code> (capitolul 5.5).
<code>relation "product" does not exist</code>	Nu s-au rulat migrările (capitolul 5.5).
caseta roșie pe pagina Acasă	Backend-ul nu este pornit sau nu răspunde la http://localhost:8080/api/health .
<code>daw-postgres</code> se oprește singur, cu eroare de permisiuni	Posibil pe Windows cu WSL 2. În <code>backend/compose.yaml</code> , urmați instrucțiunile din comentariu (volum Docker în locul folderului).
<code>Please tell me who you are</code>	Nu s-a făcut configurarea Git (capitolul 5.1).
<code>403 la git push</code>	Nu aveți încă acces în proiect (capitolul 5.4).
<code>destination path ... already exists</code>	Proiectul este deja clonat; intrați în el cu <code>cd daw-template-ifr2-2026-2027</code> .
<code>src refsPEC popescu.ion does not match any</code>	Branch-ul nu există sau numele este scris altfel; verificați cu <code>git branch</code> .

Pentru orice altă eroare de backend, citiți ultimele linii din `backend/app/var/log/dev.log` sau rulați `docker compose -f backend/compose.yaml logs -f` (ieșire cu Ctrl + C).

Anexa C. Un proiect nou, de la zero

Template-ul vă scutește de configurare, dar este util să știți cum se creează un proiect de la zero. Pentru aceasta se instalează pe laptop:

- **pentru Symfony:** PHP 8.5 (<https://windows.php.net/download/>, varianta *VS17 x64 Non Thread Safe, Zip*, dezarhivată în `C:\php`), Composer (<https://getcomposer.org/download/>), Scoop (<https://scoop.sh/>) și apoi, în PowerShell, `scoop install symfony-cli`. Un proiect nou se creează cu `symfony new numele_proiectului`;
- **pentru React:** Node.js 24 LTS (<https://nodejs.org/en/download>), apoi `npm create vite@latest numele_proiectului -- --template react`, `cd numele_proiectului`, `npm install`, `npm run dev`.

Documentație: <https://symfony.com/doc/current/setup.html> · <https://vite.dev/guide/>

Anexa D. Tutoriale și documentație

Tutoriale video

Subiect	Link
Docker, introducere (Programming with Mosh)	https://www.youtube.com/watch?v=pTFZFxd4hOI
Docker, curs complet (TechWorld with Nana)	https://www.youtube.com/watch?v=3c-iBn73dDE
Docker Compose (TechWorld with Nana)	https://www.youtube.com/watch?v=SXwC9fSwct8
Git, introducere (Programming with Mosh)	https://www.youtube.com/watch?v=8JJ101D3knE
Git, în limba română (Marius Ciurea)	https://www.youtube.com/watch?v=iVpuVkVeBcw
Git și GitHub, curs 2026 (freeCodeCamp)	https://www.youtube.com/watch?v=mAFoROnOfHs
Symfony, curs complet (freeCodeCamp; Symfony 5, concepte valabile)	https://www.youtube.com/watch?v=Bo0guUbL5uo
Symfony 7, instalare și primii pași (Coding with TD)	https://www.youtube.com/watch?v=pIGR_OPZwuQ
React, curs complet (Dave Gray)	https://www.youtube.com/watch?v=RVFAyFWO4go
React, curs (freeCodeCamp)	https://www.youtube.com/watch?v=bMknfKXIFA8
React, în limba română (playlist)	https://www.youtube.com/playlist?list=PLZuysB_DjFARX5kWZGu8hvBbmw-3VWd8w
DBeaver și PostgreSQL	https://www.youtube.com/watch?v=GgODmMBVUD4
Postman (freeCodeCamp)	https://www.youtube.com/watch?v=VywxIQ2ZXw4

Documentație oficială

- Symfony: <https://symfony.com/doc/current/index.html> · rute: <https://symfony.com/doc/current/routing.html> · controllere: <https://symfony.com/doc/current/controller.html> · Doctrine: <https://symfony.com/doc/current/doctrine.html> · migrări: <https://symfony.com/bundles/DoctrineMigrationsBundle/current/index.html>

- **NelmioApiDocBundle (Swagger):**
<https://symfony.com/bundles/NelmioApiDocBundle/current/index.html> · **NelmioCorsBundle:**
<https://symfony.com/bundles/NelmioCorsBundle/current/index.html> · **CORS explicat:**
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS>
- **React:** <https://react.dev/learn> · **React Router:** <https://reactrouter.com/> · **Vite:**
<https://vite.dev/guide/>
- **Docker:** <https://docs.docker.com/get-started/> · **Docker Compose:**
<https://docs.docker.com/compose/>
- **PostgreSQL:** <https://www.postgresql.org/docs/current/>
- **Git:** <https://git-scm.com/docs> · **GitHub, branch-uri:** <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-branches>